K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

School of Engineering and Technology

ADA LAB SHEET - 1

COURSE CODE - ENCS256

NAME : Aryan Goyal

COURSE : B.TECH CSE(FSD)

ROLL NO : 2401350001

GITHUB LINK : assignment link

Title:

Sorting a Set of Numbers Using Various Algorithms and Analyzing Their Efficiency .

Introduction :

Sorting algorithms are fundamental in computer science, used to arrange data in a specific order to enhance efficiency and accessibility. This assignment focuses on implementing various sorting techniques to organize numbers into ascending or descending order. By utilizing the step-count method, we will calculate the time complexity of each algorithm to analyze their performance. Input data will be taken from a table and operations will be repeated with varying input sizes (10, 20, 30, 40 numbers). The study includes examining best-case, worst-case, and average-case scenarios, culminating in visualizing results through graphs to interpret computational trends effectively.

Objective:

The objective of this assignment is to implement multiple sorting algorithms, sort a given set of numbers in ascending and descending order, and analyze their performance using the step-count method. Additionally, students will examine the best-case, worst-case, and average-case scenarios and visualize the results through graphs.

CODE:

```
import java.util.Random;

public class SortingAnalysis {

    // ---------- Utility Method: Swap ----------
    private static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }

    // ---------- Bubble Sort ----------
    public static long performBubbleSort(int[] data) {
        long comparisonCount = 0;
        int n = data.length;

        for (int pass = 0; pass < n - 1; pass++) {
            boolean isSorted = true;

            for (int i = 0; i < n - pass - 1; i++) {
                comparisonCount++;
                if (data[i] > data[i + 1]) {
                    swap(data, i, i + 1);
                    isSorted = false;
                }
            }

            if (isSorted) break; // early exit
        }
        return comparisonCount;
    }

    // ---------- Selection Sort ----------
    public static long performSelectionSort(int[] data) {
        long comparisonCount = 0;
        int n = data.length;

        for (int i = 0; i < n - 1; i++) {
            int smallestIndex = i;
```

```java
        for (int j = i + 1; j < n; j++) {
            comparisonCount++;
            if (data[j] < data[smallestIndex]) {
                smallestIndex = j;
            }
        }

        if (smallestIndex != i) {
            swap(data, i, smallestIndex);
        }
    }
    return comparisonCount;
}

// ---------- Insertion Sort ----------
public static long performInsertionSort(int[] data) {
    long comparisonCount = 0;

    for (int i = 1; i < data.length; i++) {
        int current = data[i];
        int j = i - 1;

        while (j >= 0) {
            comparisonCount++;

            if (data[j] > current) {
                data[j + 1] = data[j];
                j--;
            } else {
                break;
            }
        }
        data[j + 1] = current;
    }
    return comparisonCount;
}

// ---------- Data Generators ----------
public static int[] createRandomData(int size) {
    Random random = new Random();
    int[] arr = new int[size];
```

```java
        for (int i = 0; i < size; i++) {
            arr[i] = random.nextInt(1000);
        }
        return arr;
    }

    public static int[] createSortedData(int size) {
        int[] arr = new int[size];
        for (int i = 0; i < size; i++) {
            arr[i] = i;
        }
        return arr;
    }

    public static int[] createReverseData(int size) {
        int[] arr = new int[size];
        for (int i = 0; i < size; i++) {
            arr[i] = size - i;
        }
        return arr;
    }

    // ---------- Display Helper ----------
    public static void displayResults(String caseType, int size,
                                      int[] baseArray) {

        System.out.println("\n" + caseType + " (n = " + size + ")");

        System.out.println("Bubble Sort Comparisons   : " +
                performBubbleSort(baseArray.clone()));

        System.out.println("Selection Sort Comparisons: " +
                performSelectionSort(baseArray.clone()));

        System.out.println("Insertion Sort Comparisons: " +
                performInsertionSort(baseArray.clone()));
    }

    // ---------- Main ----------
    public static void main(String[] args) {

        int[] inputSizes = {10, 20, 30, 40};
```

```
        for (int size : inputSizes) {


System.out.println("\n=====================================");
            System.out.println(" ANALYSIS FOR INPUT SIZE = " + size);

System.out.println("=====================================");

            displayResults("BEST CASE (Already Sorted)",
                    size, createSortedData(size));

            displayResults("WORST CASE (Reverse Sorted)",
                    size, createReverseData(size));

            displayResults("AVERAGE CASE (Random Data)",
                    size, createRandomData(size));
        }
    }
}
```

OUTPUT:

FOR INPUT SIZE 10:

```
=====================================
 ANALYSIS FOR INPUT SIZE = 10
=====================================

BEST CASE (Already Sorted) (n = 10)
Bubble Sort Comparisons   : 9
Selection Sort Comparisons: 45
Insertion Sort Comparisons: 9

WORST CASE (Reverse Sorted) (n = 10)
Bubble Sort Comparisons   : 45
Selection Sort Comparisons: 45
Insertion Sort Comparisons: 45

AVERAGE CASE (Random Data) (n = 10)
Bubble Sort Comparisons   : 42
Selection Sort Comparisons: 45
Insertion Sort Comparisons: 31
```

FOR INPUT SIZE 20:

```
=====================================
 ANALYSIS FOR INPUT SIZE = 20
=====================================

BEST CASE (Already Sorted) (n = 20)
Bubble Sort Comparisons   : 19
Selection Sort Comparisons: 190
Insertion Sort Comparisons: 19

WORST CASE (Reverse Sorted) (n = 20)
Bubble Sort Comparisons   : 190
Selection Sort Comparisons: 190
Insertion Sort Comparisons: 190

AVERAGE CASE (Random Data) (n = 20)
Bubble Sort Comparisons   : 187
Selection Sort Comparisons: 190
Insertion Sort Comparisons: 107
```

FOR INPUT SIZE 30:

```
=====================================
 ANALYSIS FOR INPUT SIZE = 30
=====================================

BEST CASE (Already Sorted) (n = 30)
Bubble Sort Comparisons   : 29
Selection Sort Comparisons: 435
Insertion Sort Comparisons: 29

WORST CASE (Reverse Sorted) (n = 30)
Bubble Sort Comparisons   : 435
Selection Sort Comparisons: 435
Insertion Sort Comparisons: 435

AVERAGE CASE (Random Data) (n = 30)
Bubble Sort Comparisons   : 407
Selection Sort Comparisons: 435
Insertion Sort Comparisons: 263
```

FOR INPUT SIZE 40:

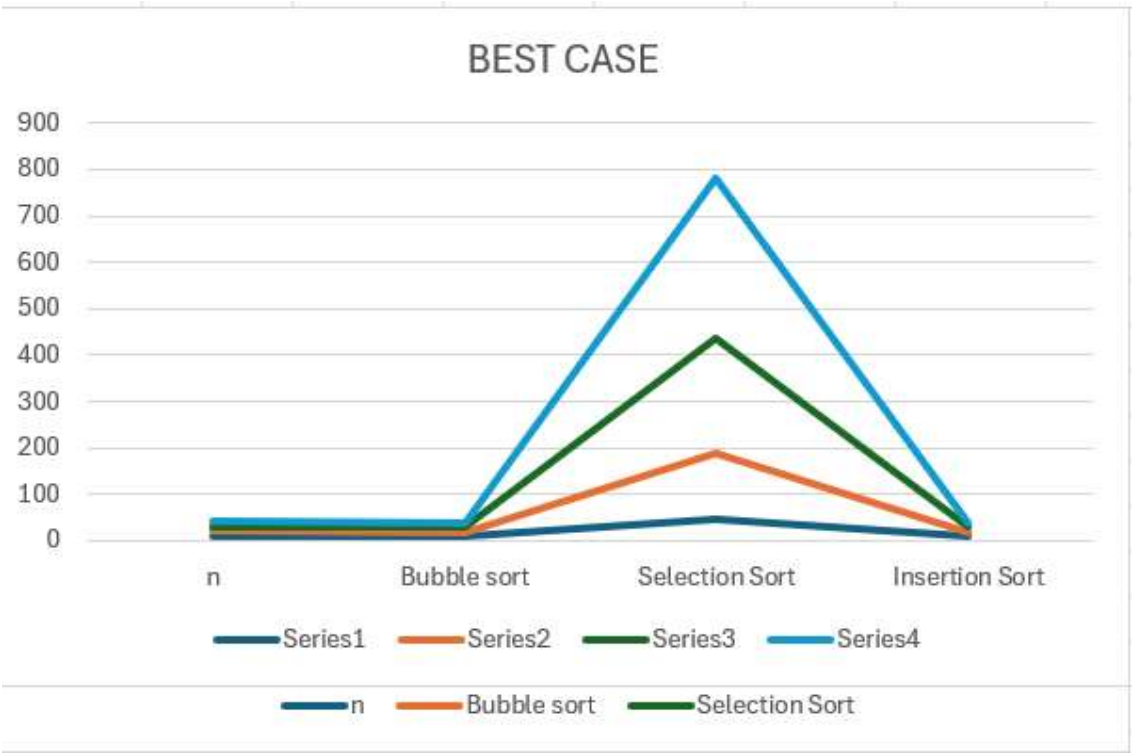
```
=====================================
 ANALYSIS FOR INPUT SIZE = 40
=====================================

BEST CASE (Already Sorted) (n = 40)
Bubble Sort Comparisons   : 39
Selection Sort Comparisons: 780
Insertion Sort Comparisons: 39

WORST CASE (Reverse Sorted) (n = 40)
Bubble Sort Comparisons   : 780
Selection Sort Comparisons: 780
Insertion Sort Comparisons: 780

AVERAGE CASE (Random Data) (n = 40)
Bubble Sort Comparisons   : 777
Selection Sort Comparisons: 780
Insertion Sort Comparisons: 493
PS C:\Users\LENOVO\OneDrive\Desktop\ADA\LAB_SHEET_1>
```

TABLES:

BEST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	9	45	9
20	19	190	19
30	29	435	29
40	39	780	39

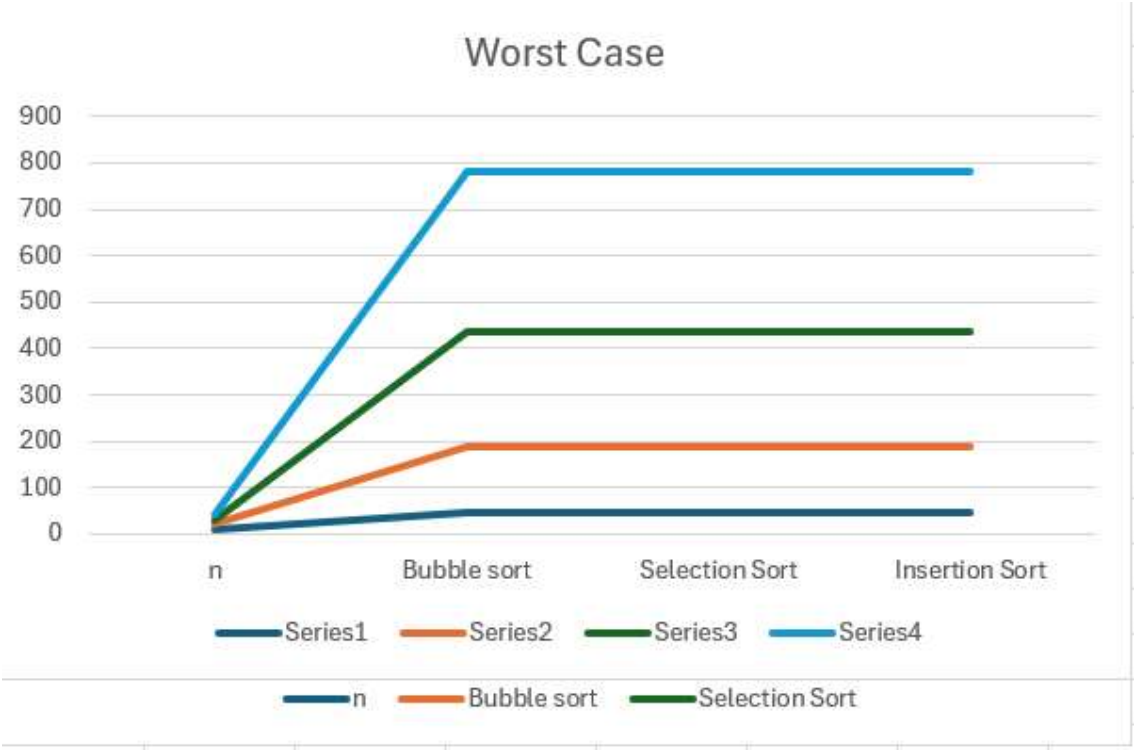
GRAPH



WORST CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	45	45	45
20	190	190	190
30	435	435	435
40	780	780	780

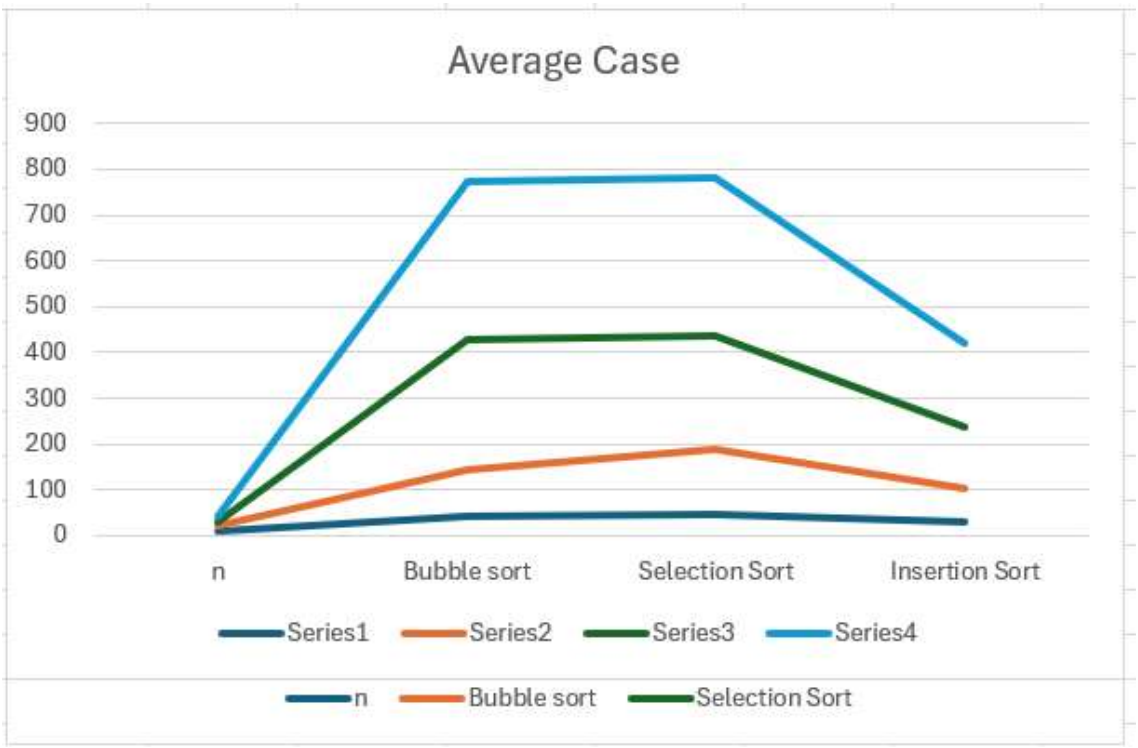
GRAPH



AVERAGE CASE

Input Size (n)	Bubble Sort	Selection Sort	Insertion Sort
10	44	45	30
20	145	190	104
30	429	435	236
40	774	780	421

GRAPH



Observation From Graphs:

From the graphs and comparison count tables, we observe that:

1. **Selection Sort** shows almost the same number of comparisons in best, average, and worst cases.
   This is because it always scans the entire unsorted portion of the array regardless of input order.
   Hence, its time complexity remains **$O(n^2)$** in all cases.

2. **Bubble Sort** performs very efficiently in the best case due to the optimization (early stopping when no swaps occur).
   Therefore, its best-case complexity becomes **$O(n)$**.
   However, in average and worst cases, it performs nearly **$O(n^2)$** comparisons.

3. **Insertion Sort** performs efficiently for already sorted data, showing linear growth in the best case (**$O(n)$**).
   In the worst case, it performs **$O(n^2)$** comparisons, similar to Bubble and Selection Sort.

4. From the average-case graph, Insertion Sort performs better than Bubble Sort for larger inputs.